Lec 4: Communication

Naveen Kumar

Assistant Professor
SVNIT, Surat

February 15, 2026

Table of Contents

1. Introduction
2. Foundations
3. Remote Procedure Call
4. Message Oriented Communication
5. Summary

Table of Contents

1. Introduction
2. Foundations
3. Remote Procedure Call
4. Message Oriented Communication
5. Summary

Communication in Distributed Systems

- Communication is the **foundation** of every distributed system
- Processes execute on different machines and must **exchange information**
- Early systems relied on **low-level message passing**
- Message passing is:
  - Harder to program than shared memory
  - Error-prone in large, unreliable networks (e.g., the Internet)
- Modern systems involve **thousands to millions of processes**
- Hence, scalable systems require **higher-level communication abstractions**

What This Chapter Covers

- **Protocols and Layering**
  - Rules governing communication between processes
  - Layering as a tool for managing complexity
- **Remote Procedure Call (RPC)**
  - Makes remote interaction resemble local procedure calls
  - Well suited for client–server architectures
  - Transparency versus performance trade-offs
- **Message-Oriented Middleware (MOM)**
  - Asynchronous, message-based communication
  - Supports loose coupling beyond client–server models
- **Multicasting**
  - Communication with multiple receivers
  - Deterministic (trees) and probabilistic (gossip, flooding) techniques

Table of Contents

1 Introduction

2 Foundations

3 Remote Procedure Call

4 Message Oriented Communication

5 Summary

Why Communication Matters

- Distributed systems have **no shared memory**
- All coordination is achieved through **message passing**
- Correct communication requires agreement on:
  - Message meaning
  - Message representation
  - Rules governing message exchange (protocols)

Process-to-Process Communication

1. Process P constructs a message in its address space
2. P invokes an operating-system communication primitive
3. The OS transmits the message across the network
4. The message arrives at the machine hosting process Q
5. Process Q receives and interprets the message

Key requirement: Sender and receiver must interpret the same bit pattern identically.

Communication Protocols

- A **protocol** precisely defines:
  - Message format
  - Message contents
  - Message semantics
- Protocols specify the services provided to higher layers
- Successful communication requires compatible protocols at both ends

Types of Communication Services

Connection-oriented

- Explicit setup before data transfer
- Parameters negotiated during setup
- Explicit connection termination
- Example: Telephone system

Connectionless

- No setup phase
- Independent message transmission
- Example: Postal mail, IP packets

Why Layered Communication?

- Communication involves many distinct concerns:
  - Physical transmission
  - Error detection and recovery
  - Routing and congestion control
  - Reliability and flow control
  - Data representation
- Layering decomposes complexity
- Each layer solves a well-defined problem

The OSI Reference Model

- Proposed by ISO as a standard reference model
- Known as the **OSI model**
- Designed to enable communication between **open systems**
- OSI protocols are rarely used today
- OSI layering remains conceptually essential

Open Systems

- An open system can interoperate with other open systems
- Based on standardized protocols
- Standards define:
  - Message format
  - Message content
  - Message interpretation

The OSI Reference Model: Layers and Responsibilities

- Network communication structured into **seven layers**:
  1. **Physical** — raw bit transmission
  2. **Data Link** — framing, error detection, retransmission
  3. **Network** — routing and congestion control
  4. **Transport** — reliability, ordering, flow control
  5. **Session** — dialog control, synchronization
  6. **Presentation** — data representation and encoding
  7. **Application** — network applications and protocols
- Transport protocols:
  - **TCP**: reliable, ordered, connection-oriented
  - **UDP**: connectionless, unreliable, lightweight

Message Encapsulation

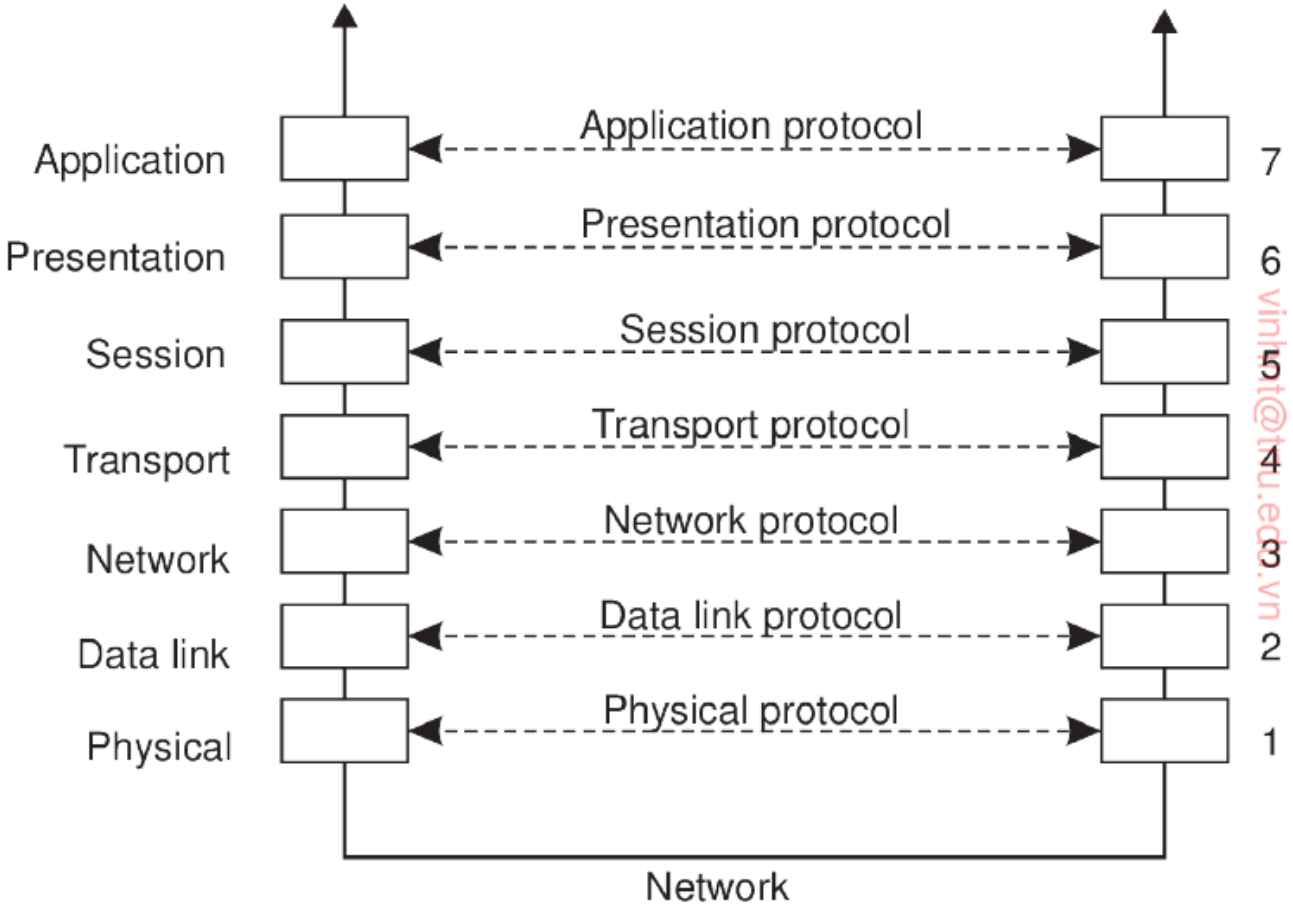


Figure 1: Layers, interfaces, and protocols in the OSI model

Protocol Stacks

- A protocol stack is an ordered collection of protocols
- OSI is a conceptual reference model
- TCP/IP is a practical protocol suite
- Real systems often merge or omit OSI layers

Middleware

- Logically positioned above the transport layer
- Provides application-independent services
- Hides heterogeneity and distribution
- Examples:
  - Naming (DNS)
  - Authentication
  - Distributed transactions
  - Distributed locking

Adapted Communication Model

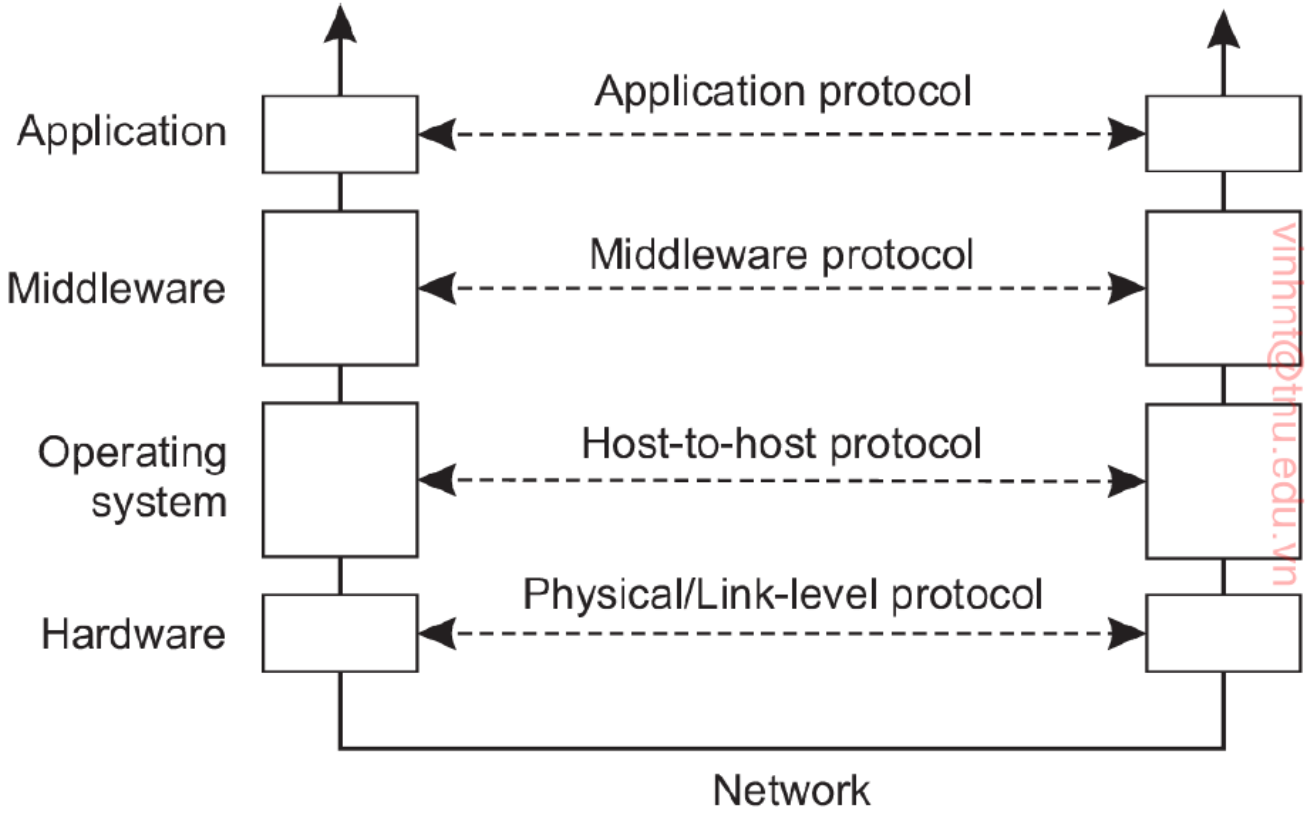


Figure 2: An adapted reference model for networked communication

Types of Communication

- Communication classified by:
  - **Persistence** (message lifetime)
  - **Synchronization** (blocking behavior)
- **Persistent communication**
  - Messages stored until delivery
  - Time-decoupled interaction
- **Transient communication**
  - Messages exist only during execution
  - Lost on failure or receiver inactivity

Synchronous vs Asynchronous Communication

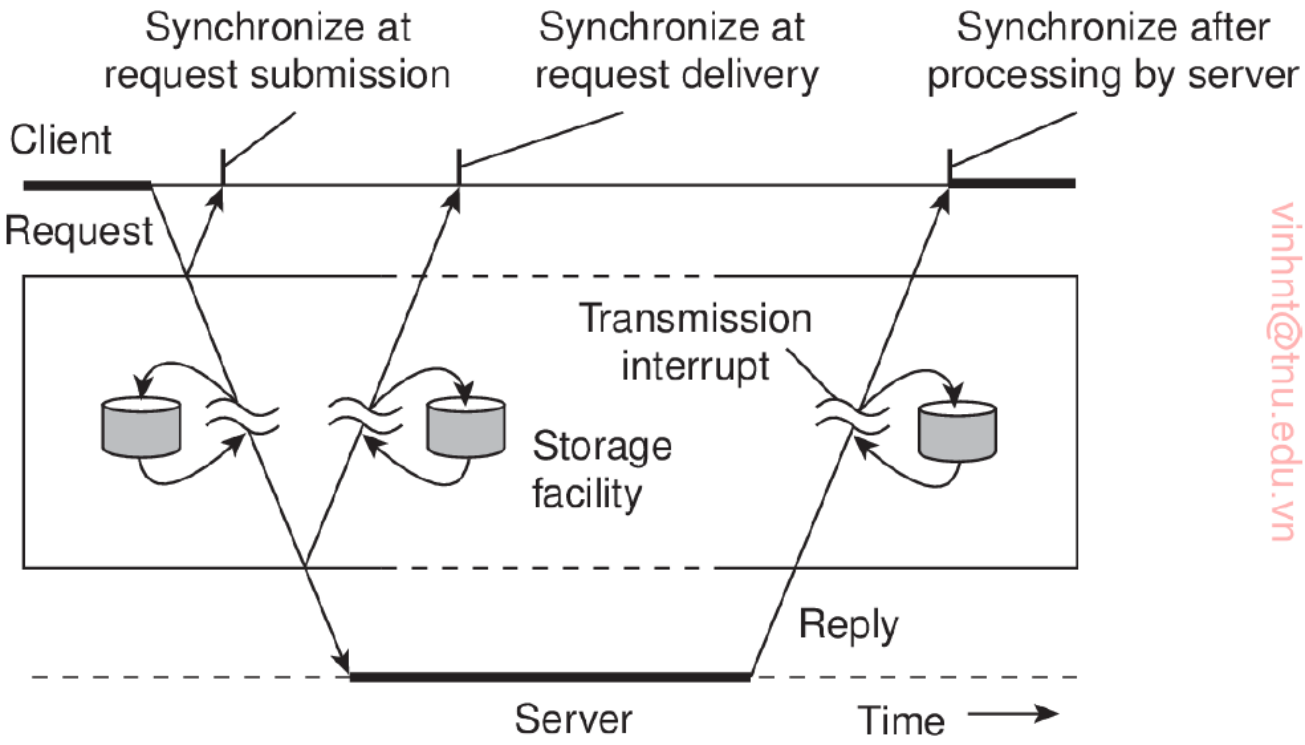
Asynchronous

- Sender continues immediately after sending

Synchronous

- Sender blocks until request is accepted, delivered, or processed

Middleware as Intermediary



vinhnt@tnu.edu.vn

Figure 3: Middleware as an intermediate service in application communication

Key Takeaways and Next Steps

- Communication is central to distributed systems
- Layering structures complexity
- Middleware bridges transport and applications
- Communication differs by persistence and synchronization
- **Next:** Remote Procedure Call (RPC)

Table of Contents

1 Introduction

2 Foundations

3 Remote Procedure Call

4 Message Oriented Communication

5 Summary

Motivation and Idea of RPC

- Early distributed systems used explicit `send` and `receive`
- Message passing exposes communication details
- This violates **access transparency**
- Programmers must manage:
  - Message construction and ordering
  - Error handling

- **RPC idea (Birrell & Nelson, 1980s):**
  Allow remote procedure calls to look like local calls

What Is a Remote Procedure Call?

- A process on machine A invokes a procedure on machine B
- The client process blocks until the result is returned
- Parameters are sent to the server
- The procedure executes remotely
- Results are returned to the client

Goal: Remote calls should be indistinguishable from local calls.

Why RPC Is Fundamentally Complex

- Client and server execute in different address spaces
- Parameters and results must be transferred across machines
- Machines may differ in:
  - Architecture
  - Byte ordering
  - Data representation
- One or both machines may crash

Example: A Local Procedure Call

```
newlist = append(data, dbList)
```

- **Single machine, single address space**
- `append` is linked into the executable
- Call sequence:
  - Caller pushes parameter representations onto the stack
  - Control jumps to `append`
  - Return value placed in caller context
- All memory addresses are meaningful to both caller and callee

Local Procedure Call Stack

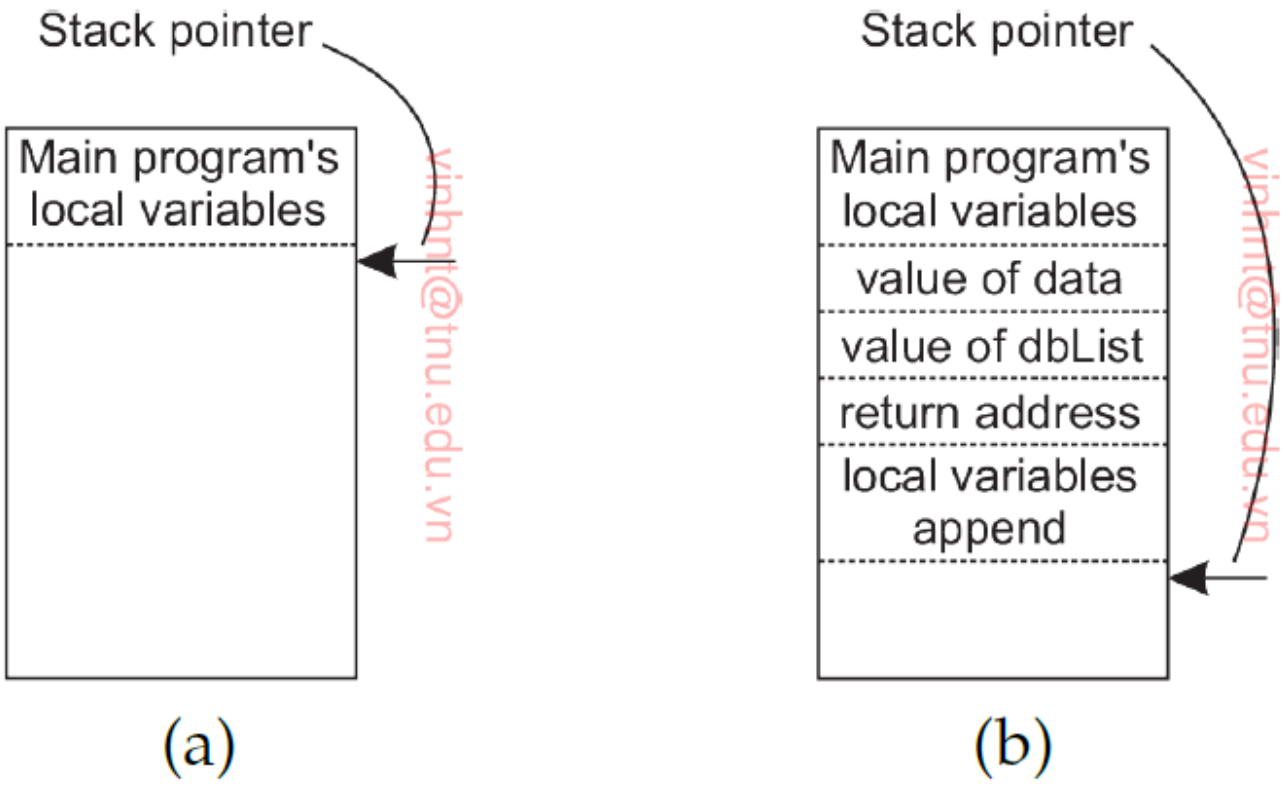


Figure 4: (a) Parameter passing in a local procedure call: the stack before the call to append. (b) The stack while the called procedure is active

vinhnt@tnu.edu.vn

Parameter Passing in Local Calls

- Parameters are passed as *representations*, not abstractions
- **Call-by-value (e.g., `data`)**
  - Actual value copied onto the stack
  - Callee works on a local copy
  - Modifications do **not** affect caller
- **Call-by-reference (e.g., `dbList`)**
  - Memory address (pointer) is passed
  - Callee accesses the same object as caller
  - Modifications **directly affect** caller state

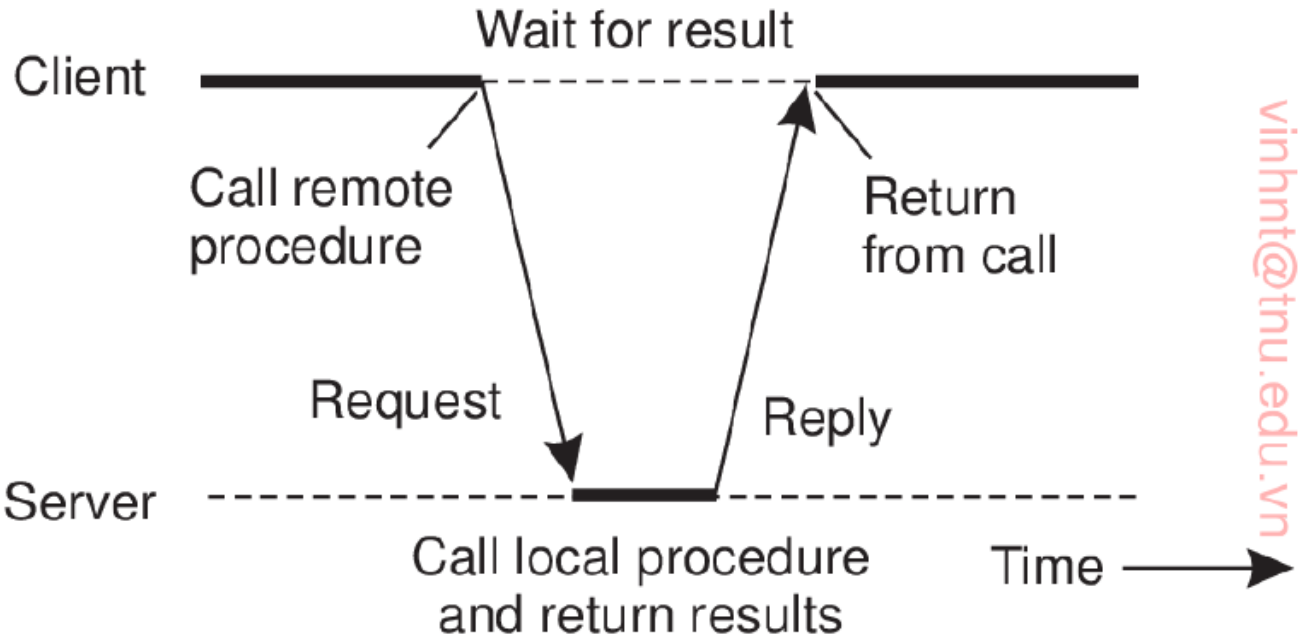
Why Local Call Semantics Do Not Carry Over to RPC

- Pointers are only meaningful within one address space
- A memory address on machine A is meaningless on machine B
- Call-by-reference cannot be preserved directly
  - Must be replaced by copying data
  - Changes semantics compared to local calls
- Leads to fundamental RPC issues:
  - Parameter marshaling
  - Copy vs reference semantics
  - Consistency and failure handling

Client and Server Stubs

- RPC replaces the real procedure with a **client stub**
- Client stub:
  - Packs parameters into a message
  - Sends request to server
  - Waits for reply
- Server stub:
  - Unpacks parameters
  - Calls local server procedure
  - Packs result and sends reply

RPC Architecture



vinhnt@tnu.edu.vn

Figure 5: The principle of RPC between a client and server program.

Steps in a Remote Procedure Call

1. The client procedure calls the **client stub** in the normal way
2. The client stub builds a message and calls the **local operating system**
3. The client's OS sends the message to the **remote OS**
4. The remote OS delivers the message to the **server stub**
5. The server stub unpacks the parameters and calls the **server procedure**
6. The server executes the procedure and returns the result to the stub
7. The server stub packs the result into a reply message and calls its **local OS**
8. The server's OS sends the reply message to the **client's OS**
9. The client's OS delivers the reply to the **client stub**
10. The client stub unpacks the result and returns it to the client procedure

RPC Execution Steps

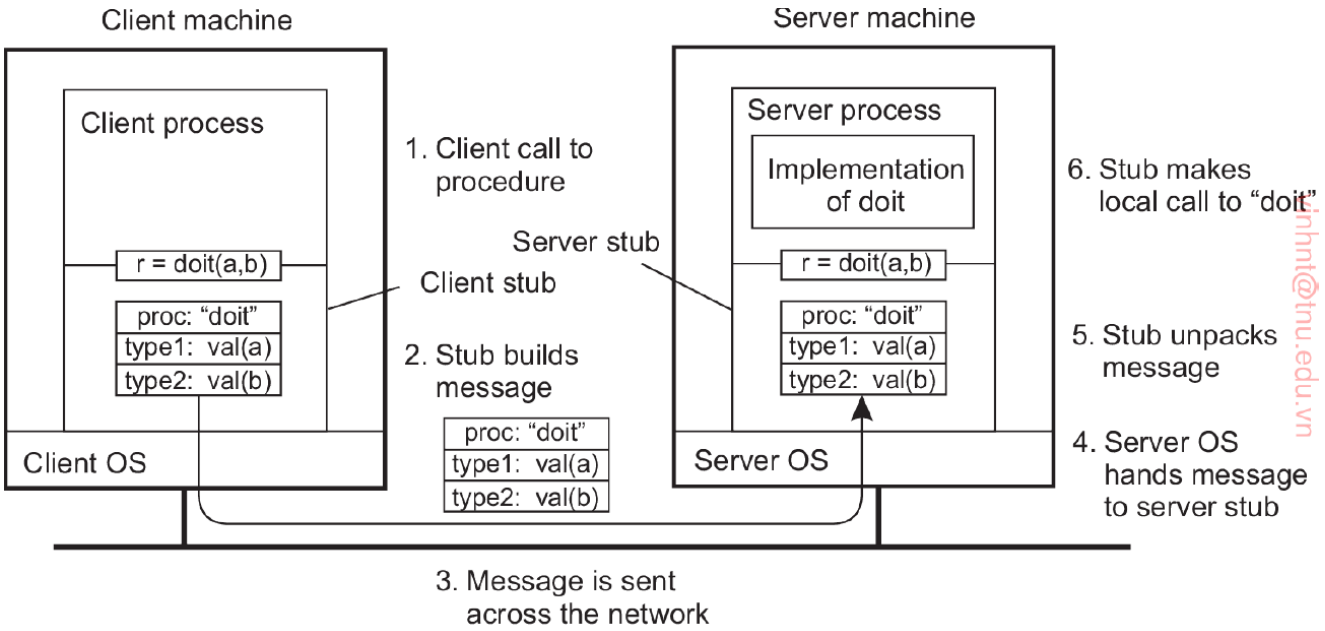


Figure 6: The steps involved in calling a remote procedure doit(a,b). The return path for the result is not shown.

tinhnt@tnu.edu.vn

RPC Code

```
import channel, pickle

class DBList:
  def append(self, data):
    self.value = self.value + [data]
    return self

class Client:
  def append(self, data, dbList):
    msglst = (APPEND, data, dbList)          # message payload
    self.chan.sendTo(self.server, msglst)    # send msg to server
    msgrcv = self.chan.recvFrom(self.server) # wait for response
    return msgrcv[1]                         # pass it to caller

class Server:
  def append(self, data, dbList):
    return dbList.append(data)

  def run(self):
    while True:
      msgreq = self.chan.recvFromAny() # wait for any request
      client = msgreq[0]               # see who is the caller
      msgrpc = msgreq[1]               # fetch call & parameters
      if APPEND == msgrpc[0]:          # check what is being requested
        result = self.append(msgrpc[1], msgrpc[2]) # do local call
        self.chan.sendTo([client],result)          # return response
```

Figure 7: A simple RPC example for operation append

Marshaling and Unmarshaling

- **Marshaling:** The process of packaging procedure parameters into a message format suitable for transmission.
  - *Example:* The client stub takes integers *a* and *b* and converts them into a byte stream: `[proc: "doit", val(a), val(b)]`.
- **Unmarshaling:** The process of unpacking the message at the receiver to store the data back into its original data types.
  - *Example:* The server stub receives the byte stream and extracts the values to call the local `Implementation of doit`.

Data Representation: Endianness

- **Problem:** Different architectures store multibyte data (like integers) differently in memory.
- **Big-Endian:** Most significant byte (MSB) stored at the lowest address (e.g., IBM, Sun Sparc).
- **Little-Endian:** Least significant byte (LSB) stored at the lowest address (e.g., Intel x86).
- **Example:** The value `0x12345678`
  - Big-Endian: [12] [34] [56] [78]
  - Little-Endian: [78] [56] [34] [12]
- **Solution:** Convert to **Network Byte Order** (standardized as Big-Endian) during marshaling.

The Problem with Pointers

- **Isolated Address Spaces:** A pointer is just a memory address (e.g., `0x004F`) valid only within the client's process.
- **Meaningless Transfer:** Passing `0x004F` to a server is useless because the server's memory at that address contains unrelated data or is inaccessible.
- **Distributed Memory:** There is no shared memory in a standard RPC; the "stack" and "heap" are physically separate.

Solutions to Passing References

- **Copy/Restore (Call-by-Value-Result):**
  - Copy the entire referenced object to the server.
  - Server modifies it and sends the whole object back to overwrite the client's local copy.
- **Serialization:** Flatten complex structures (linked lists, trees) into a contiguous byte stream.
- **Remote Object References:** Use a unique global ID that the server can use to "call back" the client (standard in Java RMI).
- **Restriction:** Simply forbid passing pointers in the Interface Definition Language (IDL).

Object References

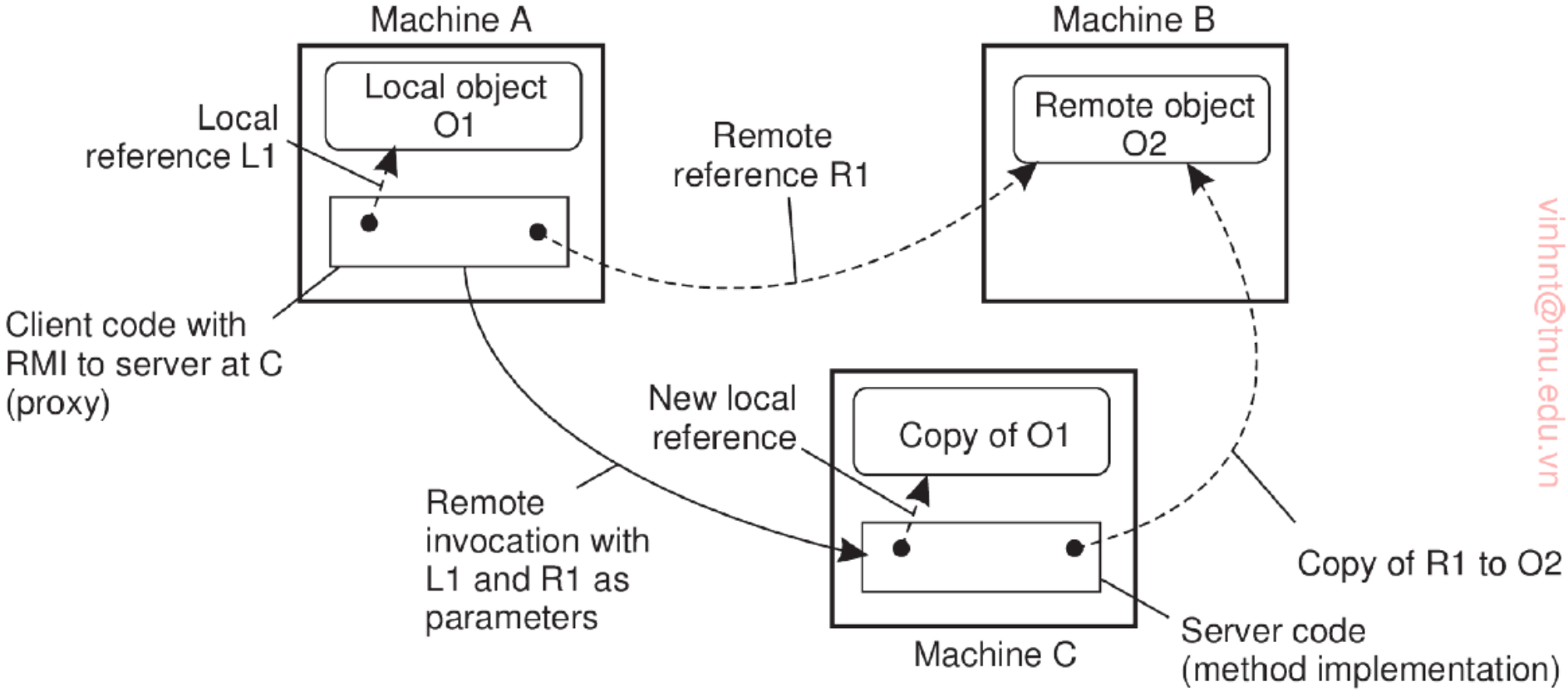


Figure 8: Passing an object by reference or by value.

vinhnt@tnu.edu.vn

RPC-Based Application Support

- RPC hides remote communication from applications
- Caller and callee must:
  - Agree on message format
  - Follow the same calling protocol
- Without a common protocol, RPC fails

Key idea: RPC correctness depends on strict agreement between both sides.

Supporting RPC-Based Applications

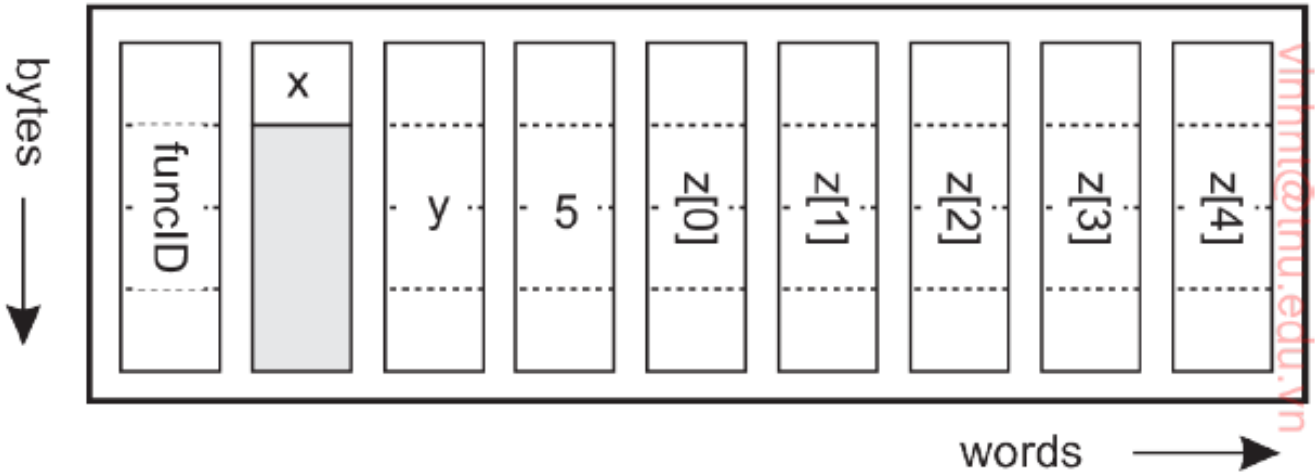
There are two main approaches:

1. **Stub generation**
   - Developer specifies remote procedures
   - Client and server stubs are generated automatically
2. **Language-level RPC support**
   - RPC embedded in the programming language
   - Remote calls look like local calls

Stub Generation and Message Format

```
void someFunction(char x; float y; int z[5])
```

(a)



(b)

Figure 9: (a) A function. (b) The corresponding message, and the order in which bytes and words are sent across the network.

vinhnt@tnu.edu.vn

Language-Based RPC

- Remote Procedure Call can be embedded into a language
- Simplifies distributed application development
- Improves access transparency
- Remote calls appear the same as local calls

Example: Java Remote Method Invocation

- Java embeds RPC as **Remote Method Invocation (RMI)**
- A client can invoke methods on remote objects
- Method calls may look local in the source code

Result: High transparency in distributed systems.

Embedding RPCs in a Programming Language

```python
import rpyc
from rpyc.utils.server import ForkingServer

class DBList(rpyc.Service):
  value = []

  def exposed_append(self, data):
    self.value = self.value + [data]
    return self.value

  def exposed_value(self):
    return self.value

if __name__ == "__main__":
  server = ForkingServer(DBList, port = 12345)
  server.start()
```

Figure 10: (a) Embedding RPCs in a language: a server

```python
import rpyc

class Client:
  conn = rpyc.connect(SERVER, PORT) # Connect to the server
  conn.root.exposed_append(2)       # Call an exposed operation,
  conn.root.exposed_append(4)       # and append two elements
  print conn.root.exposed_value()   # Print the result
```

Figure 11: (b) Embedding RPCs in a language: a client

Variations on Remote Procedure Call

- Traditional RPC uses strict request–reply
- Client blocks until the server returns a reply
- This can reduce efficiency in some situations

Motivation for variations:

- No result needs to be returned
- Client should continue execution
- Multiple RPCs should run in parallel

Asynchronous RPC

- Client does not block waiting for the result
- Server immediately sends an acknowledgment
- Server executes the procedure locally afterward
- Client continues execution after acknowledgment

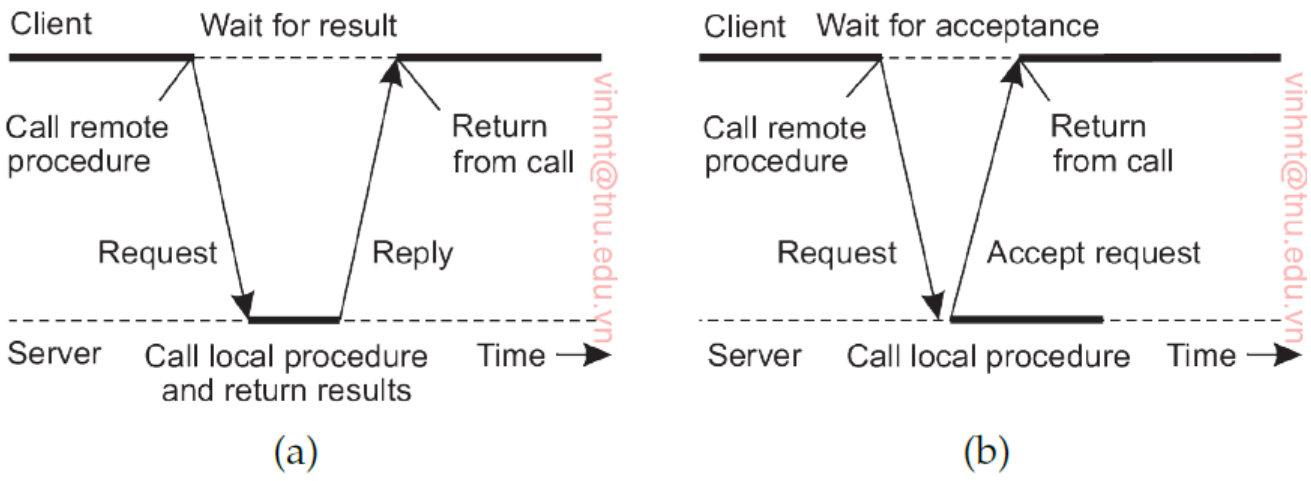


Figure 12: (a) The interaction between client and server in a traditional RPC. (b) The interaction using asynchronous RPC.

Deferred Synchronous and One-Way RPC

Deferred Synchronous RPC

- Client requires a result, but cannot wait
- Client sends request and waits only for acceptance
- Server processes request asynchronously
- Result is returned later via a **callback**

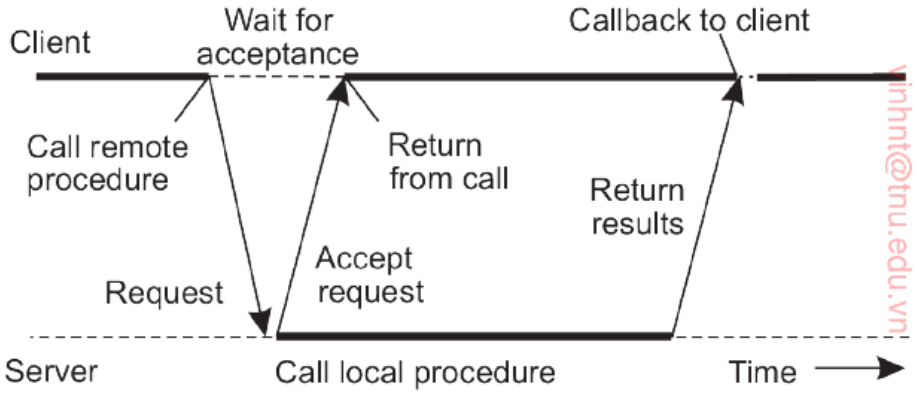


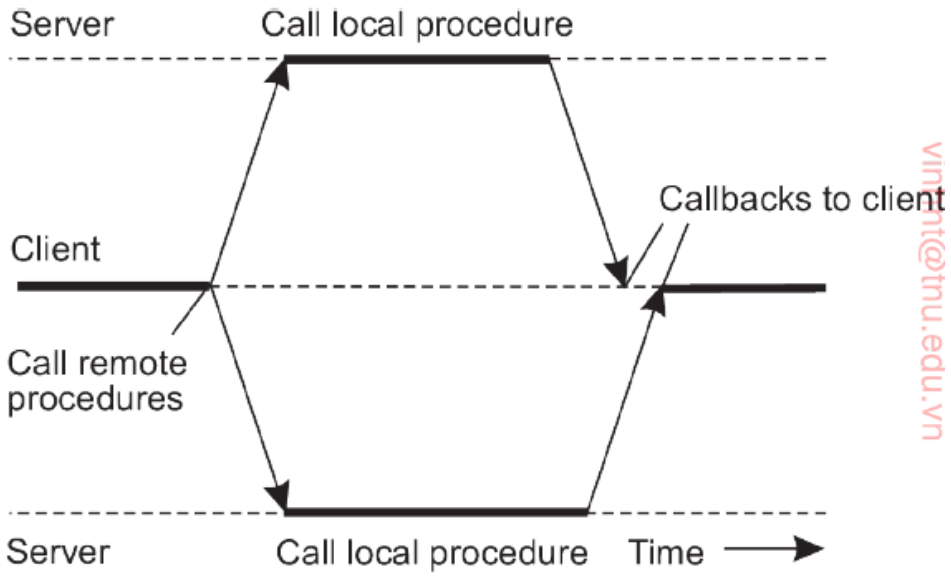
Figure 13: A client and server interacting through asynchronous RPCs.

One-Way RPC

- Client does not require a result
- Client sends request and continues immediately
- No acknowledgment and no callback
- Delivery and execution are not guaranteed

Multicast RPC

- Executes multiple RPCs simultaneously
- Client sends one request to a group of servers
- Servers process the request independently
- Processing occurs in parallel



vinnnt@tnu.edu.vn

Figure 14: The principle of a multicast RPC.

Multicast RPC: Handling Responses

- Client may wait for:
  - First response (fault tolerance)
  - Majority of responses
  - All responses (result merging)
- Servers may be replicated:
  - For fault tolerance
  - For parallel computation
- Replication can be hidden by client-side stubs

Note: Application developer must specify the purpose of multicast RPC.

Example: DCE RPC

- Distributed Computing Environment (DCE)
- Developed by OSF (now The Open Group)
- Widely used as middleware for RPC-based systems
- Basis for:
  - **Microsoft DCOM:** Enables object-based remote method invocation across Windows machines using RPC.
  - **Samba:** Allow non-Windows systems to access Windows file and print services.
- Representative of traditional RPC systems

DCE Model and Purpose

- Middleware layer between operating system and applications
- Originally designed for Unix, later ported to major OSs
- Existing (non-distributed) applications remain unaffected
- Client–server programming model
- All communication performed via RPC

Goals and Transparency of DCE RPC

- Remote service accessed using a local procedure call
- Enables reuse of large volumes of existing code
- RPC system hides:
  - Server location and discovery
  - Binding and communication setup
  - Message transport, fragmentation, and reassembly
  - Data representation and byte ordering differences
- Supports heterogeneous languages, platforms, and protocols

Interface Definition and Stub Generation

- Interfaces specified using Interface Definition Language (IDL)
- IDL defines procedure syntax, not semantics
- Shared global variables between client and server are not allowed
- Each interface has a globally unique identifier (UUID)
  - 128-bit value generated using `uuidgen`
  - Encodes creation time and location
- IDL compiler generates:
  - Header file
  - Client stub
  - Server stub

DCE RPC Development Steps

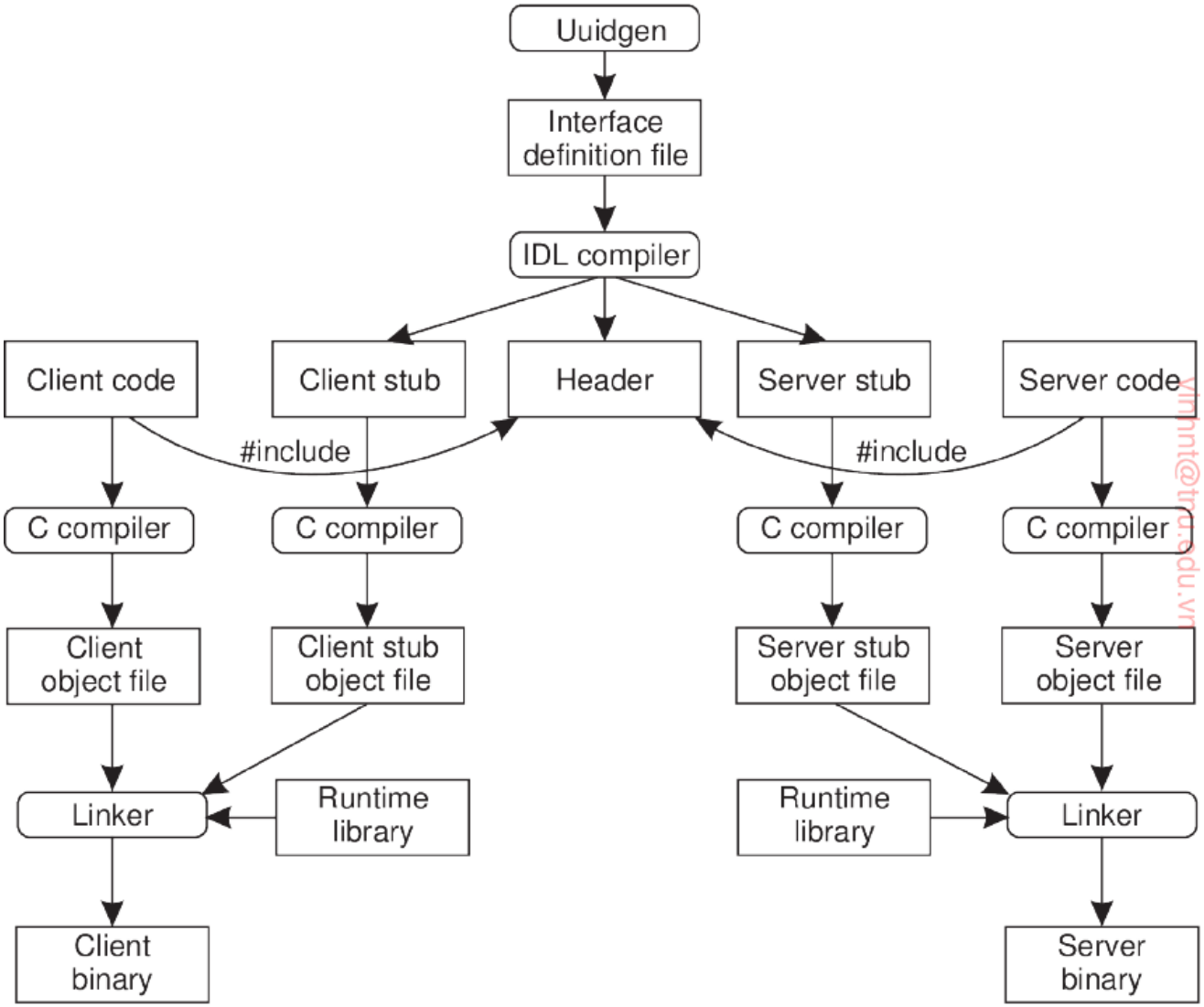


Figure 15: The steps in writing a client and a server in DCE RPC.

DCE RPC: Interface and Stub Generation

DCE RPC starts with defining a **remote interface**. A UUID is generated to uniquely identify this interface.

The programmer writes an **IDL file**. It lists remote procedures and their parameters. Only syntax is defined, not behavior.

The **IDL compiler** processes the IDL file. It generates:

- A shared header file
- A client stub
- A server stub

These stubs hide all communication details.

DCE RPC: Building and Running an RPC

Client code is compiled with the client stub. Server code is compiled with the server stub. Both include the same header file.

Object files are linked with the **RPC runtime library**. This produces the client and server binaries.

At run time, the client calls a local procedure. The client stub sends a request to the server. The server stub calls the real server code. The result is returned transparently.

RPC makes remote calls look local.

Binding a Client to a Server

- Server startup:
  - Requests a port from the OS
  - Registers (server, port) with the DCE daemon
  - Registers a logical name with the directory service
- Client binding:
  - Queries directory service using server name
  - Contacts DCE daemon to obtain server port
- Binding performed once and reused for subsequent RPCs

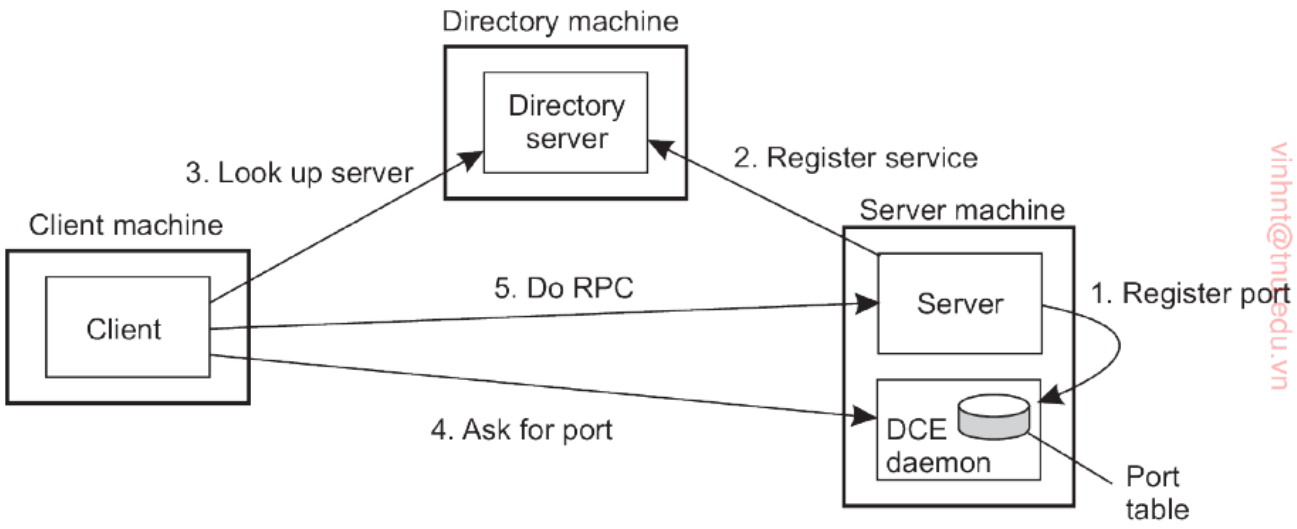


Figure 16: Client-to-server binding in DCE.

vinhnt@tnu.edu.vn

Performing an RPC and Semantics

- Client stub marshals parameters
- Runtime system transmits request using chosen protocol
- Server stub unmarshals parameters and invokes server procedure
- Reply returned along the reverse path
- Secure RPC supported for confidentiality and integrity

Table of Contents

1 Introduction

2 Foundations

3 Remote Procedure Call

4 Message Oriented Communication

5 Summary

Why Message-Oriented Communication?

- RPC/RMI improve **access transparency** by hiding communication
- However, RPC implicitly assumes:
  - Receiver is executing
  - Client blocks until reply (synchronous)
- In real distributed systems, these assumptions may not hold
  - Components may be temporarily unavailable
  - Blocking reduces scalability
- **Message-oriented systems** adopt asynchronous communication
  - Sender proceeds without waiting
  - Looser coupling between sender and receiver

Roadmap of Messaging Solutions

To address the limitations of RPC, we categorize message-oriented systems into two strategies:

Type	**Meaning**	**Solves "Blocking"?**	**Solves "Availability"?**
Transient	Data is discarded if it cannot be delivered immediately.	**Yes.** Sender proceeds without waiting for a reply.	**No.** Receiver must be active to accept the data.
Persistent	Data is stored in a queue for as long as it takes to deliver.	**Yes.** Decouples the sender from the receiver's timeline.	**Yes.** Messages are stored even if the receiver is offline.

- **Transient Examples:** Sockets, ZeroMQ, MPI.
- **Persistent Examples:** IBM WebSphere MQ, AMQP (RabbitMQ).

Sockets and TCP Operations

Socket Basics

- Socket: standardized communication end point over TCP/UDP.
- Provides transport abstraction with `send()` / `recv()` interface.

TCP Communication Workflow

Server:

- `socket()` → `bind()`
- `listen()` → `accept()`

Client:

- `socket()` → `connect()`
- Data exchange

Operation	Description
`socket`	Create a new communication end point
`bind`	Attach a local address to a socket
`listen`	Tell operating system what the maximum number of pe connection requests should be
`accept`	Block caller until a connection request arrives
`connect`	Actively attempt to establish a connection
`send`	Send some data over the connection
`receive`	Receive some data over the connection
`close`	Release the connection

Figure 17: TCP socket operations and their semantics

Socket Communication Pattern

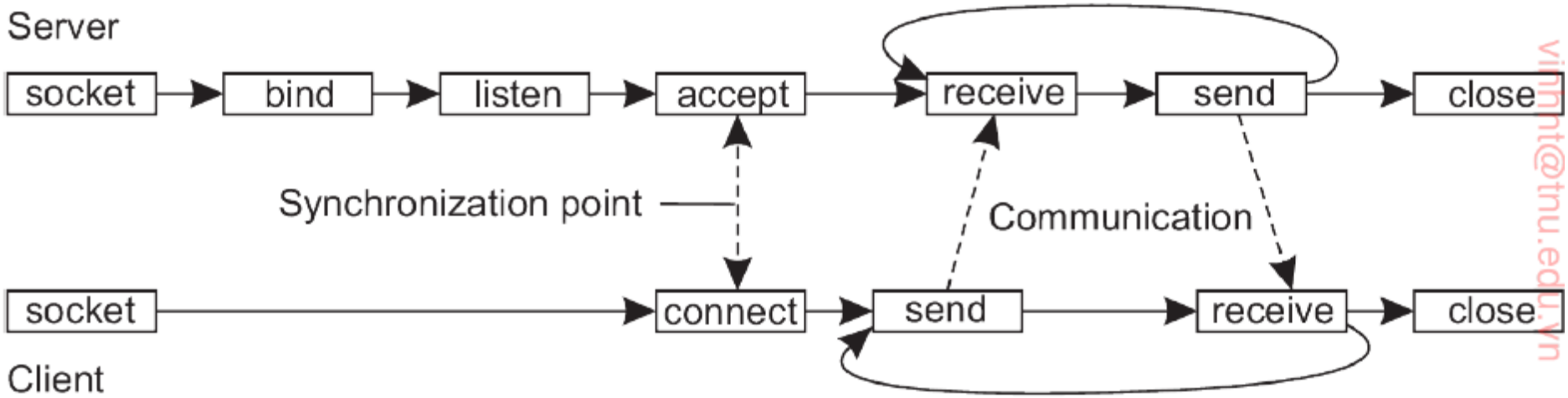


Figure 18: Connection-oriented socket communication

Socket Client–Server Example

- Server waits for incoming connections
- Client initiates connection
- Messages exchanged over TCP
- Connection closed after communication

Socket Client Server Code

```
from socket  import *
s = socket(AF_INET, SOCK_STREAM)
s.bind((HOST, PORT))
s.listen(1)
(conn, addr) = s.accept()  # returns new socket and addr. client
while True:                # forever
  data = conn.recv(1024)   # receive data from client
  if not data: break       # stop if client stopped
  conn.send(str(data)+"*") # return sent data plus an "*"
conn.close()               # close the connection
```

Figure 19: Simple socket-based server code

```
from socket  import *
s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT)) # connect to server (block until accepted)
s.send('Hello, world')  # send same data
data = s.recv(1024)     # receive the response
print data              # print the result
s.close()               # close the connection
```

Figure 20: Simple socket-based client code

Stubs as Global References

- A **stub** hides message passing from applications
- In advanced messaging, a stub itself can be:
  - Marshaled
  - Transmitted
  - Used by another process
- The stub now acts as a **global reference**
- Method calls on the stub transparently invoke the server

Client-Side Stub: DBClient

- `DBClient` is a **client-side stub**
- Represents a remote list managed by the server
- Provides high-level operations:
  - `create`
  - `getValue`
  - `appendData`
- All operations internally call `sendrecv`
- `sendrecv`:
  - Opens a socket
  - Sends a marshaled request
  - Receives and unmarshals the reply

```
class DBClient:
  def sendrecv(self, message):
    sock = socket()                          # create a socket
    sock.connect((self.host, self.port))     # connect to server
    sock.send(pickle.dumps(message))         # send some data
    result = pickle.loads(sock.recv(1024)) # receive the response
    sock.close()                             # close the connection
    return result

  def create(self):
    self.listID = self.sendrecv([CREATE])
    return self.listID

  def getValue(self):
    return self.sendrecv([GETVALUE, self.listID])

  def appendData(self, data):
    return self.sendrecv([APPEND, data, self.listID])
```

```
def __init__(self, host, port):
    self.host = host  # Stores the server IP/Hostname
    self.port = port  # Stores the server Port
    self.listID = None # Initialized after create() is called
```

Figure 21: (a) Client-side stub: DBClient

Server-Side List Server

- Server manages multiple lists
- Lists stored in `setOfLists` indexed by list IDs
- Server workflow:
  - Accept incoming socket connection
  - Unmarshal request
  - Execute operation locally
  - Marshal and send result
- Uses standard socket pattern: `socket` → `bind` → `listen` → `accept`

```
class Server:
  def __init__(self, port=PORT):
    self.host = 'localhost'                # this machine
    self.port = port                       # the port it will listen to
    self.sock = socket()                   # socket for incoming calls
    self.sock.bind((self.host,self.port)) # bind socket to an address
    self.sock.listen(5)                    # max num of connections
    self.setOfLists = {}                   # init: no lists to manage

  def run(self):
    while True:
      (conn, addr) = self.sock.accept() # accept incoming call
      data = conn.recv(1024)            # fetch data from client
      request = pickle.loads(data)      # unwrap the request
      if request[0] == CREATE:                # create a list
        listID = len(self.setOfLists) + 1     # allocate listID
        self.setOfLists[listID] = []          # initialize to empty
        conn.send(pickle.dumps(listID))       # return ID

      elif request[0] == APPEND:              # append request
        listID = request[2]                   # fetch listID
        data   = request[1]                   # fetch data to append
        self.setOfLists[listID].append(data) # append it to the list
        conn.send(pickle.dumps(OK))           # return an OK

      elif request[0] == GETVALUE:            # read request
        listID = request[1]                   # fetch listID
        result = self.setOfLists[listID]      # get the elements
        conn.send(pickle.dumps(result))       # return the list
      conn.close()                      # close the connection
```

Figure 22: (b)Server-side list server

Client for Passing Stubs

- This `Client` is an application-level communicator
- It is **not** the list stub
- Responsibilities:
  - Listen for incoming connections
  - Send arbitrary objects to other clients
- Uses marshaling to transmit:
  - Data
  - Stubs (such as `DBClient`)

```
class Client:
  def __init__(self, port):
    self.host = 'localhost'                # this machine
    self.port = port                       # port it will listen to
    self.sock = socket()                   # socket for incoming calls
    self.sock.bind((self.host, self.port)) # bind socket to an address
    self.sock.listen(2)                    # max num connections

  def sendTo(self, host, port, data):
    sock = socket()
    sock.connect((host, port))    # connect to server (blocking call)
    sock.send(pickle.dumps(data)) # send some data
    sock.close()

  def recvAny(self):
    (conn, addr) = self.sock.accept()
    return conn.recv(1024)
```

Figure 23: (c) Client for sending/receiving stubs

Passing Stubs as Global References

- Client 1:
  - Creates a list on the server
  - Appends data
  - Sends the `DBClient` stub to Client 2
- Client 2:
  - Receives and unmarshals the stub
  - Uses it like a local object
  - Appends data to the same remote list
- Both clients operate on the **same remote object**

```
pid = os.fork()
if pid == 0:
  client1   = Client(CLIENT1)                # create client
  dbClient1 = DBClient(HOST,PORT)            # create reference
  dbClient1.create()                         # create new list
  dbClient1.appendData('Client 1')           # append some data
  client1.sendTo(HOSTCL2,CLIENT2,dbClient1) # send to other client

pid = os.fork()
if pid == 0:
  client2   = Client(CLIENT2)                # create a new client
  data      = client2.recvAny()              # block until data is sent
  dbClient2 = pickle.loads(data)             # receive reference
  dbClient2.appendData('Client 2')           # append data to same list
```

Figure 24: Passing stubs as references

Limitations of Basic Sockets

- **Low-level & Error-prone:** Developers must manually manage IP addresses, ports, and connection states.
- **Byte-Stream vs. Message:** No built-in message boundaries; the application must manually track where data ends.
- **Manual Marshaling:** Requires explicit "pickling" and "unpickling" for every data exchange.
- **Rigid Topology:** Limited support for advanced communication patterns like **Multicasting** or **Load Balancing**.
- **Tight Coupling:** Blocking calls (`accept`, `recv`) reduce overall system scalability.
- **Brittleness:** If a server is unavailable during `connect()`, the system often fails without built-in recovery.

ZeroMQ: High-Level Messaging

- **Smart Abstraction:** Replaces manual byte-stream handling with a message-based model.
- **Asynchronous by Default:** Senders proceed immediately; ZeroMQ handles background queuing and delivery.
- **Managed Connections:** Automatically handles `bind`, `connect`, and `reconnect` logic internally.
- **Decoupled Execution:** Communication can start even if the recipient process is not yet running.
- **Native Patterns:** Built-in support for **Many-to-One** and **One-to-Many** messaging topologies.

ZeroMQ Communication Patterns

- **Request–Reply:** Standard client–server communication with strict alternation, similar to RPC.
- **Publish–Subscribe:** One-to-many distribution where subscribers receive messages based on specific tags.
- **Pipeline (Push–Pull):** Parallel task distribution where workers pull workloads from one or more producers.

ZeroMQ: Request–Reply Pattern

The Request–Reply pattern is the most basic messaging model in ZeroMQ, designed to mimic traditional client–server interactions.

- **Socket Roles:**
  - **REQ (Request):** Used by the **Client** to initiate a conversation by sending a message.
  - **REP (Reply):** Used by the **Server** to receive requests and send back specific responses.
- **Strict Alternation:** ZeroMQ enforces a lock-step protocol where a `send` must be followed by a `recv` (and vice versa); out-of-order calls will result in an error.
- **RPC Semantics:** Like a Remote Procedure Call, the client blocks after sending until the server provides a response.
- **Dynamic Pairing:** Unlike raw sockets, a `REP` socket can handle multiple `REQ` clients automatically, returning each reply to the correct requester.

ZeroMQ: Request–Reply Code

- **Server Setup:** Creates a `REP` socket and binds to both `p1` and `p2` to listen on multiple ports simultaneously.
- **Server Loop:** Blocks at `recv()` until a message arrives from any bound port.
- **Client Initiation:** Creates a `REQ` socket, connects to `p1`, and sends `"Hello world 1"`.
- **Processing:**
  - Server receives data → checks for `"STOP"`.
  - If no `"STOP"`, it appends `"*"` and sends it back to the specific client.
- **Termination:** Client receives the reply, then sends `"STOP"` to break the server's infinite loop.

```
Server Side Output
#(Server starts and waits at s.recv())
#Received: "Hello world 1" -> Sending: "Hello world 1*"
#Received: "STOP" -> Server breaks loop and terminates

Clint Side Output
#Hello world 1*
```

```python
import zmq
context = zmq.Context()

p1 = "tcp://"+ HOST +":"+ PORT1 # how and where to connect
p2 = "tcp://"+ HOST +":"+ PORT2 # how and where to connect
s  = context.socket(zmq.REP)    # create reply socket

s.bind(p1)                      # bind socket to address
s.bind(p2)                      # bind socket to address
while True:
  message = s.recv()            # wait for incoming message
  if not "STOP" in message:     # if not to stop...
    s.send(message + "*")       # append "*" to message
  else:                         # else...
    break                       # break out of loop and end
```

Figure 25: (a) ZeroMQ request–reply server

```python
import zmq
context = zmq.Context()

p1 = "tcp://"+ HOST +":"+ PORT1 # how and where to connect
s  = context.socket(zmq.REQ)    # create request socket

s.connect(p1)                   # block until connected
s.send("Hello world 1")         # send message
message = s.recv()              # block until response
s.send("STOP")                  # tell server to stop
print message                   # print result
```

Figure 26: (b) ZeroMQ request–reply client

ZeroMQ: Publish–Subscribe Pattern

The Publish–Subscribe pattern enables one-to-many communication, decoupling the sender from the receivers.

- **Socket Roles:**
  - **PUB (Publisher):** Broadcasts messages to all connected endpoints simultaneously.
  - **SUB (Subscriber):** Connects to publishers to listen for specific data streams.
- **Topic-Based Filtering:** Subscribers use prefixes (tags) to filter incoming traffic; messages not matching the tag are discarded by the middleware before reaching the application.
- **Loose Coupling:**
  - **Space:** Publishers don't know who or how many subscribers exist.
  - **Time:** New subscribers can join or leave the network at any time.
- **Transient Nature:** Messages are not queued for offline subscribers; if no one is listening when a message is published, it is lost forever.

Publish–Subscribe Code (ZeroMQ)

This pattern implements a one-to-many broadcast system with automated message filtering.

- **Publisher Logic (Top):**
  - **Setup:** Creates a `PUB` socket and **binds** to a fixed port.
  - **Tagging:** Every message is prefixed with the tag `"TIME "`.
  - **Broadcast:** Periodically pushes the current system time to all connected clients.
- **Subscriber Logic (Bottom):**
  - **Setup:** Creates a `SUB` socket and **connects** to the publisher's address.
  - **The Filter:** `setsockopt` is critical; it tells ZeroMQ to only accept messages starting with the `"TIME"` prefix.
  - **Passive Receipt:** Only matching messages reach the `recv()` call; others are dropped.
- **Middleware Filtering**—The ZeroMQ library discards non-matching messages *before* they are delivered to the application, saving CPU cycles and memory.

```python
import zmq, time

context = zmq.Context()
s = context.socket(zmq.PUB)        # create a publisher socket
p = "tcp://"+ HOST +":"+ PORT      # how and where to communicate
s.bind(p)                          # bind socket to the address
while True:
  time.sleep(5)                    # wait every 5 seconds
  s.send("TIME " + time.asctime()) # publish the current time
```

Figure 27: (a) Publisher code (PUB socket)

```python
import zmq

context = zmq.Context()
s = context.socket(zmq.SUB)          # create a subscriber socket
p = "tcp://"+ HOST +":"+ PORT        # how and where to communicate
s.connect(p)                         # connect to the server
s.setsockopt(zmq.SUBSCRIBE, "TIME")  # subscribe to TIME messages

for i in range(5):  # Five iterations
  time = s.recv()   # receive a message
  print time
```

Figure 28: (b) Subscriber code (SUB socket)

Pipeline (Push–Pull) Pattern

- **Dynamic Task Distribution:** A producer (**Farmer**) generates work and pushes it into the pipeline.
- **On-Demand Pulling: Workers** pull tasks only when they are idle, preventing slow workers from becoming bottlenecks.
- **Automatic Load Balancing:** ZeroMQ ensures the first available worker receives the next task, distributing load fairly across the pool.
- **Task Parallelism:** Ideal for distributed workloads where independent tasks can be processed simultaneously across multiple nodes.

Pipeline Code (ZeroMQ)

- **Farmer Logic (PUSH):**
  - Creates a PUSH socket and **binds** to an address.
  - **Workload Generation:** Uses `random.randint` to simulate tasks with varying complexity.
  - **Queueing:** Tasks are queued at the sender side until a worker is ready.
- **Worker Logic (PULL):**
  - Creates a PULL socket and **connects** to multiple farmers (p1, p2).
  - **Fair Distribution:** Automatically receives work from the first available source.
  - **Simulation:** Uses `time.sleep` proportional to the workload to mimic real processing time.
- **Key Semantics:** Decouples work generation from work execution.

```python
import zmq, time, pickle, sys, random

context = zmq.Context()
me  = str(sys.argv[1])
s   = context.socket(zmq.PUSH)        # create a push socket
src = SRC1  if me == '1' else SRC2    # check task source host
prt = PORT1 if me == '1' else PORT2   # check task source port
p   = "tcp://"+ src +":"+ prt         # how and where to connect
s.bind(p)                             # bind socket to address

for i in range(100):                  # generate 100 workloads
  workload = random.randint(1, 100)   # compute workload
  s.send(pickle.dumps((me,workload))) # send workload to worker
```

Figure 29: Farmer: Generates & Pushes Tasks

```python
import zmq, time, pickle, sys

context = zmq.Context()
me = str(sys.argv[1])
r  = context.socket(zmq.PULL)     # create a pull socket
p1 = "tcp://"+ SRC1 +":"+ PORT1   # address first task source
p2 = "tcp://"+ SRC2 +":"+ PORT2   # address second task source
r.connect(p1)                     # connect to task source 1
r.connect(p2)                     # connect to task source 2

while True:
  work = pickle.loads(r.recv())   # receive work from a source
  time.sleep(work[1]*0.01)        # pretend to work
```

Figure 30: Worker: Pulls & Processes Tasks

MPI: Parallel Computing Model

MPI provides a high-level, hardware-independent standard for tightly-coupled parallel applications.

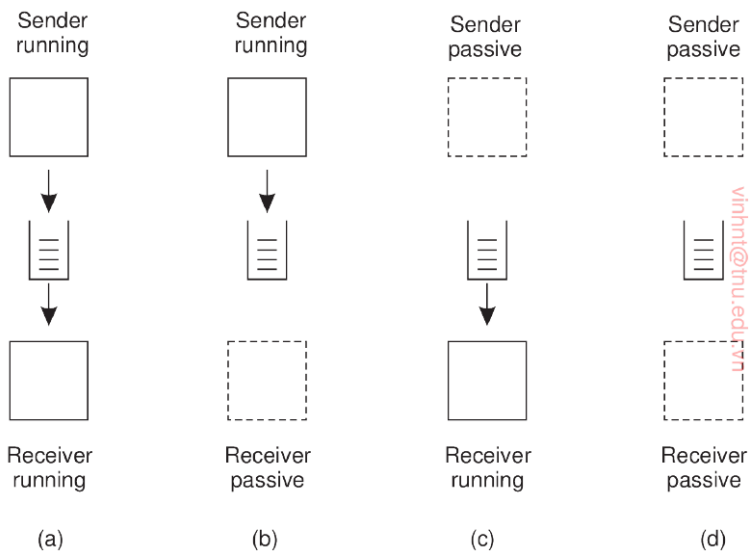
- **Why MPI?**: Sockets are too low-level; MPI is optimized for specialized interconnects and performance over general TCP/IP.
- **Addressing**: Uses `(groupID, processID)` instead of IP addresses; failures are considered **fatal**.
- **Flexible Semantics**: Supports varying performance needs:
  - **Blocking**: `MPI_send` / `MPI_recv` (wait for buffer/data).
  - **Non-blocking**: `MPI_isend` / `MPI_irecv` (immediate return).
  - **Synchronous**: `MPI_ssend` (wait until transmission starts).

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Persistent Messaging: The Queuing Model

Queuing systems enable communication even if parties are temporarily inactive.

- **Core Concept**: Apps communicate via **Queue Managers** using logical names.
- **Time-Decoupled**: Senders/receivers can be active/passive in 4 combinations.
- **Simple Interface**:
  - `put`: Non-blocking append to queue.
  - `get`: Block until queue is non-empty.
  - `poll/notify`: Non-blocking check or handler.



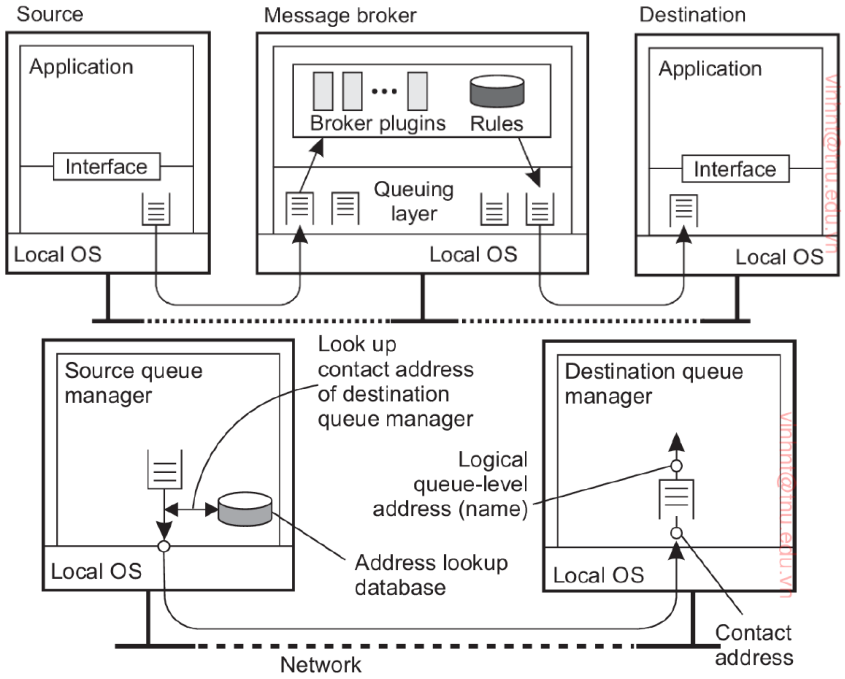
vinhnt@tnu.edu.vn

Operation	Description
`put`	Append a message to a specified queue
`get`	Block until the specified queue is nonempty, and remove the first message
`poll`	Check a specified queue for messages, and remove the first. Never block
`notify`	Install a handler to be called when a message is put into the specified queue

Message Brokers and Integration

Message-oriented middleware acts as the "glue" for heterogeneous distributed systems.

- **Message Brokers**: Application-level gateways that convert message formats and semantics.
- **Architecture**: Uses a queuing layer and rules-based "plugins" for routing.
- **Naming**: Logical queue names are mapped to physical network addresses.
- **Applications**: Enterprise integration, workflow systems, and batch processing.



Example: IBM WebSphere MQ

- Enterprise message-queuing system by IBM
- Formerly known as MQSeries
- Now part of IBM WebSphere
- Widely used in enterprise integration
- Illustrates practical message-queuing systems

WebSphere MQ: Architecture Overview

- Messages stored and forwarded via **queues**
- Queues managed by **queue managers**
- Queue manager responsibilities:
  - Remove messages from send queues
  - Forward messages to other queue managers
  - Receive messages and store them in input queues
- Supports large messages (MBs) and large queues (GBs)

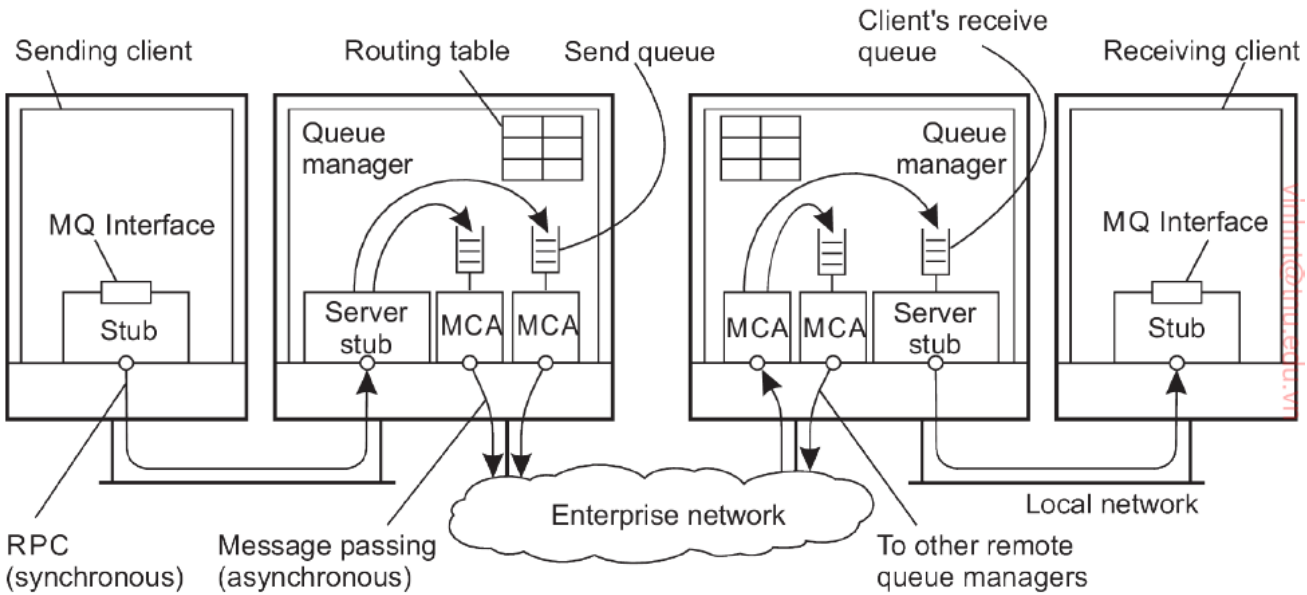


Figure 31: General organization of IBM's message-queuing system.

Message Channels and MCAs

- Queue managers connected by **message channels**
- Channel = unidirectional, reliable connection
- Each channel endpoint managed by a:
  - Sending MCA
  - Receiving MCA
- Sending MCA:
  - Fetches messages from send queue
  - Wraps and transmits them
- Receiving MCA:
  - Unwraps messages
  - Stores them in destination queue

Channel Management and Attributes

- Channels active only when both MCAs are running
- MCA startup options:
  - Manual startup
  - Triggered by first message arrival
  - Remote startup via control message
- Channels stop automatically after inactivity
- Channels configured using attributes

Attribute	Description
Transport type	Determines the transport protocol to be used
FIFO delivery	Indicates that messages are to be delivered in the order they are sent
Message length	Maximum length of a single message
Setup retry count	Maximum number of retries to start up the remote MCA
Delivery retries	Maximum times MCA will try to put received message into queue

Figure 32: Some attributes associated with message channel agents.

Message Routing, Aliases, and MQI

- Each message carries:
  - Destination queue manager name
  - Destination queue name
- Routing via local **send queues**
- Queue managers use routing tables (aliases)
- Aliases decouple applications from physical destinations
- Message may traverse multiple queue managers

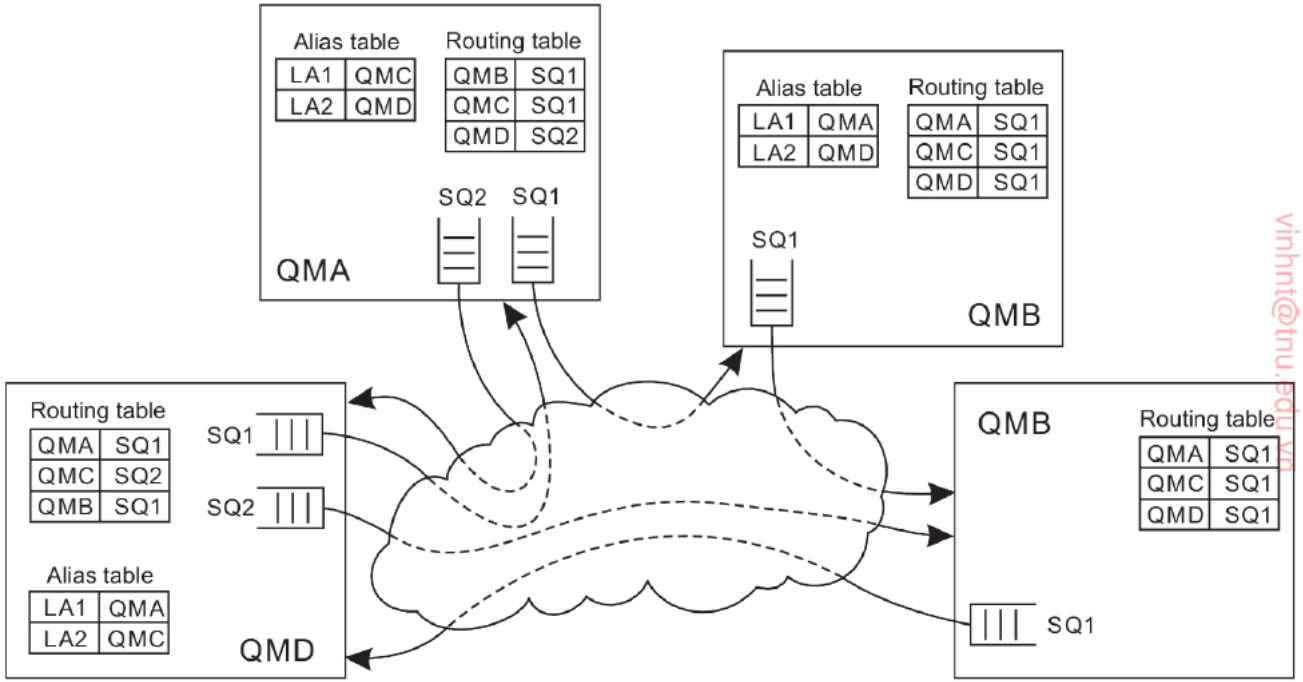


Figure 33: The general organization of an MQ queuing network using routing tables and

vinhnt@tnu.edu.vn

MQ Programming Interface and Management

- Applications use the Message Queue Interface (MQI)
- Core operations:
  - `MQOPEN`, `MQCLOSE`
  - `MQPUT`, `MQGET`
- Messages removed by priority, then FIFO
- Supports notifications on message arrival
- Persistence is per-message, not per-queue
- Overlay network managed manually

Operation	Description
MQOPEN	Open a (possibly remote) queue
MQCLOSE	Close a queue
MQPUT	Put a message into an opened queue
MQGET	Get a message from a (local) queue

Figure 34: Operations available in the message-queuing interface.

Example: Advanced Message Queuing Protocol (AMQP)

- Standardized message-queuing protocol
- Designed to overcome proprietary MQ solutions
- Developed by an industry working group (since 2006)
- Current standard: AMQP 1.0
- Popular implementations:
  - RabbitMQ
  - Apache Qpid

Motivation and Scope of AMQP

- Message-queuing systems are largely proprietary
- Interoperability between vendors is difficult
- AMQP aims to provide:
  - Open standard
  - Vendor-independent messaging
- Multiple AMQP versions coexist
- Pre-1.0 and 1.0 versions are incompatible

AMQP Architecture and Components

- Three conceptual layers:
  - Messaging service
  - Messaging protocol
  - Messaging interface
- Core components:
  - Applications
  - Queue managers
  - Queues
- Applications interact via an AMQP stub

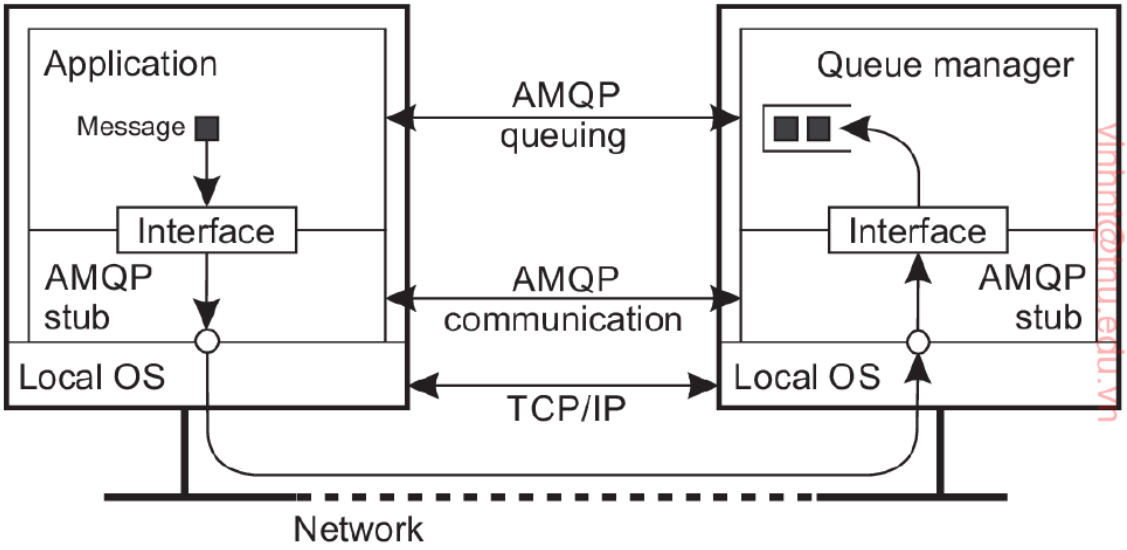


Figure 35: An overview of a single-server AMQP instance.

AMQP Communication Model

- Communication built hierarchically:
  - **Connection**: long-lived, TCP-based
  - **Channel**: lightweight, one-way
  - **Session**: bidirectional (pair of channels)
  - **Link**: message transfer endpoint
- Multiple channels multiplexed over one TCP connection
- Flow control implemented using credits

Reliable Message Transfer in AMQP

- Message transfer proceeds in three phases:
  - Message sent and marked *unsettled*
  - Receiver processes message and reports status
  - Sender and receiver mark message as *settled*
- Supports true end-to-end reliability
- Applications can delay settlement if needed

Messaging Semantics and Persistence

- Messaging entities:
  - Producers
  - Consumers
  - Queues
- Receiver may accept or reject messages
- Supports fragmentation and reassembly
- Persistence achieved through:
  - Durable messages
  - Durable nodes
- AMQP defines messaging, not higher-level patterns
- Overlay routing managed externally

Table of Contents

1 Introduction

2 Foundations

3 Remote Procedure Call

4 Message Oriented Communication

5 Summary

Summary: Communication in Distributed Systems

- Communication is fundamental to all distributed systems
- Transport-layer messaging offers low-level send/receive primitives
- Middleware raises abstraction to simplify interprocess communication
- Goal: make communication easier, safer, and less error-prone

Summary: RPC and Message-Oriented Communication

- Remote Procedure Call (RPC):
  - Service implemented as server-side procedure
  - Client sees only procedure signature
  - Client stub marshals parameters into messages
  - Server executes procedure and returns results
  - Client stub unmarshals reply transparently
- RPC provides synchronous communication:
  - Client blocks until reply arrives
  - Relaxed variants exist, but synchrony remains limiting
- Message-oriented models often better suit loosely coupled systems

Summary: Messaging Models and Multicasting

- Message-oriented communication dimensions:
  - Persistent vs. transient messaging
  - Asynchronous vs. synchronous communication
- Message-oriented middleware:
  - Typically persistent and asynchronous
  - Sender and receiver need not be active simultaneously
  - Widely used for large-scale system integration
- Multicast communication disseminates data to many receivers:
  - Tree-based multicast: efficient but failure-sensitive
  - Flooding: robust but resource-intensive
  - Probabilistic flooding: balances cost and reliability
  - Epidemic (gossip) protocols: simple, scalable, robust